Sketch-to-Image Generator

A Pix2Pix conditional GAN that turns line sketches into realistic images, served through a drawing web app

Track A: Generative Adversarial Networks (Unit 3) · Devansh Vashisth · November 2026 · github.com/devansh-vashisth/sketch-to-image-generator

Summary

This project implements image-to-image translation from hand-drawn line sketches to coloured photographs using Pix2Pix (Isola et al., 2017). A U-Net generator learns to fill in an outline with colour, shading and texture. A PatchGAN discriminator, conditioned on the same sketch, judges whether each image patch looks real. The model was trained on a 3,000-image subset of edges2shoes at 128 × 128 px in about 12 minutes on an RTX 3050 laptop GPU. An L1-only ablation was trained for comparison. A FastAPI web app lets a user draw a sketch in the browser and get a generated image back.

Item	Outcome
Model	U-Net generator (41.8 M params at 128 px) + 70 × 70 PatchGAN discriminator (2.8 M)
Training	3,000 pairs, 20 epochs (12 constant + 8 decay), batch 8, mixed precision, ~35 s / epoch
Pix2Pix results	L1 0.0862 · PSNR 16.51 dB · SSIM 0.695 · FID 184.0 (200 val images)
Main finding	L1-only wins on pixel metrics but is blurry; the adversarial loss gives sharper, more realistic texture, as reported in the original paper
Deliverables	Code, training and evaluation pipeline, ablation, web app, concepts PDF, slide deck, this report

1. Problem and approach

Given a sketch x, the task is to produce an image y that matches the outline and looks like a real photo. The mapping has many valid answers: a sketch does not determine colour or material. A plain regression loss such as L1 therefore averages over the valid answers and produces a blurry result. A conditional GAN fixes this by adding a learned loss: a discriminator that penalises any output that does not look real.

2. Architecture

Generator: U-Net

- Encoder of 7 strided 4 × 4 convolutions (at 128 px) that halve the resolution down to a 1 × 1 bottleneck; LeakyReLU(0.2) and BatchNorm.
- Decoder of 7 transposed convolutions back to 128 × 128 with ReLU; dropout 0.5 in the first three decoder blocks provides stochasticity.
- Skip connections concatenate each encoder feature map with the decoder map of the same size. This carries the exact edge positions across, so the output keeps the outline. Final Tanh gives an RGB image in [−1, 1].

Discriminator: 70 × 70 PatchGAN

- Input is the sketch concatenated with a real or generated image (4 channels).
- Three strided convolutions plus two stride-1 layers produce a grid of logits (14 × 14 at 128 px). Each logit scores one overlapping 70 × 70 patch as real or fake.
- Judging patches instead of the whole image focuses D on local texture and sharpness, the part L1 cannot supervise, and keeps the network small (2.8 M parameters).
- All weights are initialised from N(0, 0.02), following the paper.

3. Training objective

Generator loss:

$$L_G = \mathrm{BCE}(D(x, G(x)), 1) + \lambda_{L1} \cdot \|y - G(x)\|_1, \text{ with } \lambda_{L1} = 100$$

Discriminator loss:

$$L_D = \tfrac{1}{2} [\mathrm{BCE}(D(x, y), 1) + \mathrm{BCE}(D(x, G(x)), 0)]$$

Each iteration first updates D on one real and one generated pair, then updates G. The L1 term pins down low-frequency structure (shape, overall colour). The adversarial term supplies high-frequency detail. Halving the D loss slows the discriminator so it does not overpower the generator. The code also supports LSGAN loss, label smoothing and a VGG-16 perceptual loss.

4. Data

edges2shoes contains 50,025 training pairs built from UT-Zappos50K shoe photos with HED edge maps. Each file is 512 × 256, with the sketch on the left and the photo on the right. This run used the first 3,000 training pairs and all 200 validation pairs. Augmentation (resize, random crop, horizontal flip) is applied identically to both halves of a pair so they stay aligned. Pixel values are scaled to [−1, 1]. For other domains, scripts/prepare_dataset.py builds pairs from any folder of photos using XDoG or Canny edge extraction. At inference, user drawings are cropped, centred and binarised to resemble the training sketches.



Figure 1. edges2shoes validation pairs: sketch (top), photo (bottom).

5. Training setup

Setting	Value
Resolution	128 × 128 px (reduced from 256 to fit the time budget)
Data	3,000 train / 200 validation pairs

Batch size	8
Epochs	12 at constant learning rate, then 8 with linear decay to 0
Optimiser	Adam, learning rate 2e-4, β = (0.5, 0.999), for both G and D
Loss weights	λ_GAN = 1, λ_L1 = 100 (Pix2Pix); λ_GAN = 0 for the L1-only ablation
Normalisation	BatchNorm
Precision	Automatic mixed precision (fp16) on CUDA
Hardware / time	NVIDIA RTX 3050 6 GB laptop GPU, ~35 s per epoch, ~12 min in total
Checkpoint selection	Best validation SSIM (epoch 12 for both models)

6. Training dynamics

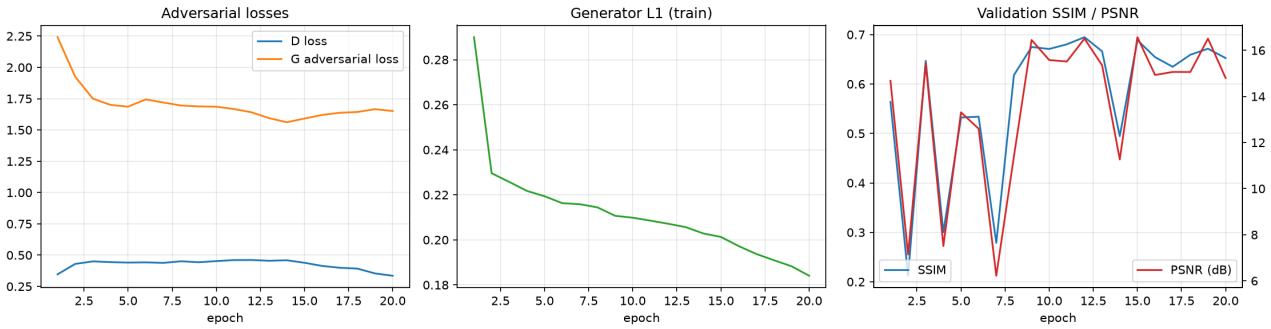


Figure 2. Pix2Pix training curves: adversarial losses, generator L1 on the training set, and validation SSIM / PSNR per epoch.

- **The adversarial game is stable.** D loss stays around 0.4–0.45 and never collapses to zero, so the generator keeps receiving useful gradients.
- **Reconstruction improves steadily.** Training L1 falls from 0.29 to 0.18, fastest once the learning rate starts to decay.
- **Validation metrics are noisy.** They swing from epoch to epoch because validation runs the generator in eval mode with BatchNorm running statistics, which are unstable with batch 8 and short training. InstanceNorm, or keeping BatchNorm in train mode at test time as the original Pix2Pix does, would fix this.



Figure 3. Generator output on four validation sketches. Columns: sketch, epochs 1, 3, 9 and 20, real photo. Shape is learned first; texture and colour variation arrive later.

7. Results

Quantitative (all 200 validation images, best-SSIM checkpoint):

Model	L1 ↓	PSNR ↑	SSIM ↑	FID ↓
L1 only (λ_GAN = 0)	0.0745	18.17 dB	0.733	176.2
Pix2Pix (cGAN + L1)	0.0862	16.51 dB	0.695	184.0



Figure 4. Qualitative comparison. Rows: input sketch, L1-only, Pix2Pix, ground truth.

Discussion

- **The pixel metrics favour L1-only, as expected.** L1, PSNR and SSIM are maximised by predicting the per-pixel mean of all plausible images. That mean is the smooth, uniform brown shoe in row 2 of Figure 4.

- **Pix2Pix trades pixel accuracy for realism.** The adversarial loss produces laces, highlights and material texture (row 3). Because this texture is plausible but not identical to the specific ground-truth photo, it lowers the pixel metrics. This matches the original paper's ablation.
- **FID is inconclusive at this sample size.** FID estimates 2048-dimensional feature statistics and needs thousands of images. With 200 it is high-variance and biased upwards, so the 8-point gap is within noise.
- **The short training leaves visible artefacts.** The Pix2Pix outputs show checkerboard patterns and colour noise, which typically fade with longer training at full resolution.

8. Web application

- **Front end** (app/static): an HTML canvas with pen, eraser, brush size, undo, clear, image upload and an optional live mode that generates after every stroke. The result is shown next to the canvas and can be downloaded as PNG.
- **Back end** (app/server.py): FastAPI. `POST /api/generate` cleans the sketch (crop, centre, binarise, resize), runs the generator on the GPU and returns a PNG. `GET /api/health` reports whether a checkpoint is loaded.
- **Verified:** the server loads the checkpoint on CUDA and returns a generated image for a validation sketch.

```
$env:S2I_CHECKPOINT="runs/pix2pix/checkpoints/best.pt"
uvicorn app.server:app --port 8000 # then open http://localhost:8000
```

9. Code structure and reproduction

Path	Purpose
`s2i/models.py`	U-Net generator, PatchGAN discriminator, weight init
`s2i/losses.py`	Adversarial loss (BCE / LSGAN), VGG-16 perceptual loss
`s2i/dataset.py`	Paired dataset (aligned or photos + on-the-fly sketch), joint augmentation
`s2i/sketch.py`	XDoG and Canny sketch extraction, user-sketch cleaning
`s2i/metrics.py`	L1, PSNR, SSIM, FID
`train.py`	Alternating D/G updates, Adam, LR decay, AMP, checkpoints, sample grids
`evaluate.py`	Validation metrics, FID and qualitative grids → metrics.json
`infer.py`	Command-line inference on a file or folder
`app/`	FastAPI server and drawing canvas
`scripts/`	Data download, dataset preparation, curve plots, comparison figure

```
python scripts/download_data.py edges2shoes
python train.py --train-dir data/edges2shoes/train --val-dir data/edges2shoes/val \
    --max-train 3000 --image-size 128 --epochs 12 --epochs-decay 8 --out runs/pix2pix
python train.py ... --lambda-gan 0 --out runs/l1_only
python evaluate.py --checkpoint runs/pix2pix/checkpoints/best.pt --data-dir
data/edges2shoes/val --fid
python scripts/plot_curves.py runs/pix2pix
python scripts/make_comparison.py --pix2pix runs/pix2pix/checkpoints/best.pt --l1
runs/l1_only/checkpoints/best.pt --data-dir data/edges2shoes/val
```

10. Limitations and next steps

Limitation	Next step
Short run: 3k images, 128 px, 20 epochs	Full schedule: 10k+ images at 256 px (about 2–3 h on the same GPU)
Checkerboard noise and colour speckle	Longer training; add perceptual loss (--lambda-vgg 10)
Noisy validation from BatchNorm in eval mode	Use --norm instance, or train-mode BN at test time
FID on 200 images is unreliable	Compute FID on at least 2,000 generated images
Single domain (shoes)	Build new domains from photo folders with XDoG (prepare_dataset.py)

Conclusion

The project delivers a complete, working sketch-to-image system, from data preparation and GAN training to evaluation and an interactive web app. Even with a small training budget, the ablation reproduces the central Pix2Pix result: an L1 loss alone gives blurry averages, and the conditional adversarial loss is what produces realistic texture. Pixel metrics alone do not capture that improvement, so qualitative comparison and perceptual metrics such as FID matter when judging generative models.

References

- Isola, Zhu, Zhou, Efros. *Image-to-Image Translation with Conditional Adversarial Networks.* CVPR 2017.
- Goodfellow et al. *Generative Adversarial Nets.* NeurIPS 2014.
- Ronneberger, Fischer, Brox. *U-Net: Convolutional Networks for Biomedical Image Segmentation.* MICCAI 2015.
- Winnemöller, Kyprianidis, Olsen. *XDoG: An eXtended Difference-of-Gaussians.* Computers & Graphics 2012.
- Heusel et al. *GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium.* NeurIPS 2017.
- Yu, Grauman. *Fine-Grained Visual Comparisons with Local Learning (UT-Zappos50K).* CVPR 2014.